GranWarm: Adaptive Dependency-Aware Prewarming for Serverless Functions Under Non-Stationary Workloads Using Fuzzy-Controlled Constrained Particle Swarm Optimization

Abstract

Serverless computing delivers elastic, event-driven execution of cloud functions without explicit infrastructure management, yet suffers from a persistent performance bottleneck: the *cold start*—latency incurred when a container must be provisioned, initialized, and loaded with dependencies before execution begins. Cold-start mitigation is fundamentally a constrained optimization problem: prewarming containers reduces latency but wastes resources on idle capacity, while aggressive reclamation conserves resources but increases user-facing latency and SLA violations. Existing approaches address fragments of this problem—static keep-alive policies, workload-prediction-based provisioning, isolated fuzzy or metaheuristic controllers, and dependency-aware scheduling—but no framework jointly addresses reasoning under workload uncertainty, exploiting inter-function dependency structure for sub-container-level resource sharing, and solving the resulting constrained multi-function allocation problem under hard resource budgets.

This paper presents **GranWarm**, a closed-loop adaptive prewarming framework that integrates three complementary mechanisms, each with a distinct architectural role. A **Mamdani-type fuzzy inference system** maps real-time workload signals—invocation intensity, traffic trend, cold-start pressure, and resource utilization—to per-function warm-capacity recommendations, enabling uncertainty-aware reasoning without requiring accurate long-horizon workload predictions. A **dependency granularity tree** decomposes the function fleet's software stacks into hierarchical, shareable layers (runtimes, libraries, function-specific code), transforming the decision space from coarse per-function container counts to fine-grained resource units that exploit inter-function overlap. A **constrained Particle Swarm Optimizer** then searches for the globally optimal allocation of warm resources across granularity-tree nodes, minimizing a composite objective of cold-start latency, cold-start frequency, idle-resource cost, and SLA-violation rate, subject to platform memory and capacity constraints. GranWarm employs a dual-timescale architecture: the fuzzy controller runs on a fast loop (5–10 s) for rapid adaptation, while the PSO optimizer runs on a slower loop (30–60 s) for periodic global re-optimization. We evaluate GranWarm on a Kubernetes/Knative testbed with 12 heterogeneous benchmark functions under five workload regimes—stable, gradual-growth, bursty, periodic, and non-stationary—and conduct a systematic six-configuration ablation study. Results demonstrate that GranWarm achieves a superior Pareto frontier on the latency–resource tradeoff, reducing P95 cold-start latency and SLA violations compared to all baselines while maintaining comparable or lower resource con-

sumption, and that each architectural component contributes measurably to overall performance.

Keywords: Serverless Computing · Cold-Start Mitigation · Fuzzy Inference System · Particle Swarm Optimization · Dependency-Aware Scheduling · Adaptive Resource Management

---

1. Introduction

1.1 The Serverless Computing Paradigm

Cloud computing has undergone a progressive shift from Infrastructure-as-a-Service (IaaS), where users manage virtual machines, through Platform-as-a-Service (PaaS), where the runtime environment is managed, to Function-as-a-Service (FaaS), where the unit of deployment is a single stateless function . In the FaaS model—commercially realized by AWS Lambda, Google Cloud Functions, Azure Functions, and open-source platforms such as Knative , OpenFaaS , and Apache OpenWhisk —developers deploy event-triggered functions without provisioning or managing servers. The platform handles container lifecycle management, automatic horizontal scaling, and crucially, *scale-to-zero*: when a function receives no invocations for a configurable period, its containers are deallocated entirely, eliminating idle-resource costs.

This elasticity has driven rapid adoption. Datadog's 2024 State of Serverless report indicates that over 70% of organizations using cloud services have adopted serverless technologies in some capacity . However, the scale-to-zero capability introduces a well-documented performance challenge that remains a primary obstacle to serverless adoption for latency-sensitive applications: the **cold start**.

1.2 Anatomy of a Cold Start

When a function is invoked and a pre-initialized container is available (a *warm start*), the request is dispatched directly, incurring only network and execution latency—typically single-digit milliseconds. When no warm container exists (a *cold start*), the platform must execute a multi-stage initialization pipeline:

$$T_{\text{cold}} = T_{\text{sched}} + T_{\text{pull}} + T_{\text{runtime}} + T_{\text{deps}} + T_{\text{init}} + T_{\text{exec}}$$

where T_{sched} is the scheduling and container-creation overhead, T_{pull} is the container image pull time, T_{runtime} is the language runtime startup (e.g., JVM, Python interpreter, Node.js V8), T_{deps} is the loading and initialization of application-level dependencies (e.g., importing NumPy, loading ML models), T_{init} is function-specific initialization code (e.g., establishing database connections, reading configuration), and T_{exec} is the actual function execution time.

Empirical measurements across major cloud providers reveal that cold-start latencies range from approximately 100 ms for lightweight Node.js functions with minimal dependencies to over 10 seconds for JVM-based functions with large dependency trees or ML model loading [6, 7, 8]. Shahrad et al. , analyzing production traces from Azure Functions, found that cold starts affected up to 40% of function invocations during low-traffic periods and contributed disproportionately to tail latency, with P99 latencies exceeding warm-start P99 by 10–50×.

For applications where end-to-end latency is critical—interactive web APIs, real-time data processing pipelines, IoT event handlers, and microservice chains where cold starts compound multiplicatively—this performance penalty can render serverless architectures unsuitable, forcing architects to either over-provision resources (negating the cost benefits of serverless) or accept degraded user experience.

1.3 The Latency–Resource Tradeoff

Cold-start mitigation is not simply a matter of "keeping containers alive longer." It is fundamentally a **constrained bi-objective optimization** between two competing goals:

Objective 1 — Minimize cold-start impact:

$$\min_{\mathbf{x}\in\mathcal{X}} \quad \alpha_1 \cdot \mathbb{E}[T_{\text{cold}}(\mathbf{x})] + \alpha_2 \cdot P(\text{cold start} \mid \mathbf{x}) + \alpha_3 \cdot P(\text{SLA violation} \mid \mathbf{x})$$

Objective 2 — Minimize resource waste:

$$\min_{\mathbf{x}\in\mathcal{X}} \quad \beta_1 \cdot M_{\text{idle}}(\mathbf{x}) + \beta_2 \cdot C_{\text{idle}}(\mathbf{x})$$

where M_{idle} and C_{idle} are idle memory and CPU consumption from pre-warmed but unused containers.

These objectives are in direct tension. Aggressive prewarming (maintaining many warm containers) minimizes cold starts but increases idle-resource costs. Aggressive reclamation (quickly deallocating idle containers) conserves resources but increases cold-start frequency and latency. The optimal operating point is workload-dependent and shifts over time.

This tradeoff is compounded by four characteristics of real serverless workloads that defeat simple heuristics:

1. **Burstiness.** Invocation rates exhibit heavy-tailed distributions with sudden, unpredictable spikes. A function averaging 5 req/s may experience bursts of 500 req/s during flash events .
2. **Non-stationarity.** Workload distributions shift continuously—diurnal cycles, weekly patterns, seasonal effects, and regime changes mean that historical statistics become stale .

3. **Heterogeneity.** Functions differ vastly in cold-start cost (0.1 s to 10+ s), memory footprint (64 MB to 3+ GB), invocation frequency (sub-second to hours), and latency sensitivity GranWarm.

4. **Resource constraints.** The platform has finite memory, CPU, and budget. Resources allocated to pre-warming one function are unavailable for others.